

Spacecraft Engines Data Pipeline

Proyecto Final – Ingeniería de Datos

Pipeline de datos batch y streaming en tiempo real sobre motores de cohetes espaciales,
desplegado en AWS real

Stan Mora

Tais Rodriguez

12 de septiembre de 2026

Índice

1. Introducción y objetivo	2
2. Fuentes de datos	2
3. Arquitectura general	2
4. Flujo Batch	3
4.1. Ingesta	3
4.2. Transformación	3
4.3. Modelo de Machine Learning	4
5. Flujo Streaming (tiempo real)	5
5.1. Simulación de telemetría	5
5.2. Procesamiento en tiempo real	5
5.3. Validación del detector de anomalías	5
6. Entorno de desarrollo local (Floci)	6
7. Despliegue en AWS real	7
8. CI/CD – Automatización con GitHub Actions	8
9. Dashboard	9
10. Tecnologías utilizadas	11
11. Conclusiones	11

1. Introducción y objetivo

Este proyecto implementa un pipeline de datos completo sobre motores de cohetes espaciales, cubriendo los dos tipos de flujo requeridos por el curso: procesamiento **batch** (histórico de lanzamientos) y procesamiento **streaming en tiempo real** (telemetría simulada de motores). Ambos flujos están desplegados sobre servicios reales de AWS – no simulaciones locales únicamente – e incluyen ingesta desde múltiples fuentes, transformación, almacenamiento, un modelo de machine learning, un detector de anomalías en tiempo real, y un dashboard interactivo que consolida ambos flujos en una sola pantalla.

El objetivo es demostrar, de punta a punta, el ciclo completo de un pipeline moderno de datos: ingesta → almacenamiento → transformación → análisis/modelado → visualización, con infraestructura real (no simulada) donde corresponde, y con desarrollo local reproducible antes de tocar la nube.

2. Fuentes de datos

El proyecto ingiere datos de **dos fuentes independientes**, cumpliendo el requisito de ingesta multi-fuente:

1. **Launch Library 2 API** (11.thespacedevs.com) – API pública, gratuita y sin necesidad de clave de autenticación, que provee el histórico real de lanzamientos espaciales de múltiples agencias: SpaceX, NASA, Roscosmos, ESA, Rocket Lab y Blue Origin. Se consulta una vez por familia de cohete conocida.
2. **Tabla estática de especificaciones de motores** (`src/batch/engine_specs.py`) – un archivo Python con especificaciones reales y públicamente documentadas (empuje, impulso específico, tipo de propelente) de 10 motores de cohetes: Merlin 1D, Rocketdyne F-1, RS-25, RD-108A, RD-180, RS-68A, Vulcain 2, Rutherford y BE-4. No proviene de una API porque no existe ninguna fuente gratuita con ese nivel de detalle por motor – son valores físicos reales, ampliamente citados públicamente.

3. Arquitectura general

La Figura 1 muestra la arquitectura completa: dos flujos paralelos e independientes que convergen en un único dashboard.

Un aspecto de diseño importante: **el mismo código corre indistintamente en local (contra Floci, un emulador de AWS tipo LocalStack ejecutado vía Docker) o contra AWS real** – el único cambio es una variable de entorno (`AWS_ENV=local` vs `AWS_ENV=aws`) en el archivo `.env`. Esto permite desarrollar y probar todo el pipeline sin costo ni riesgo antes de desplegarlo contra la cuenta real de AWS.

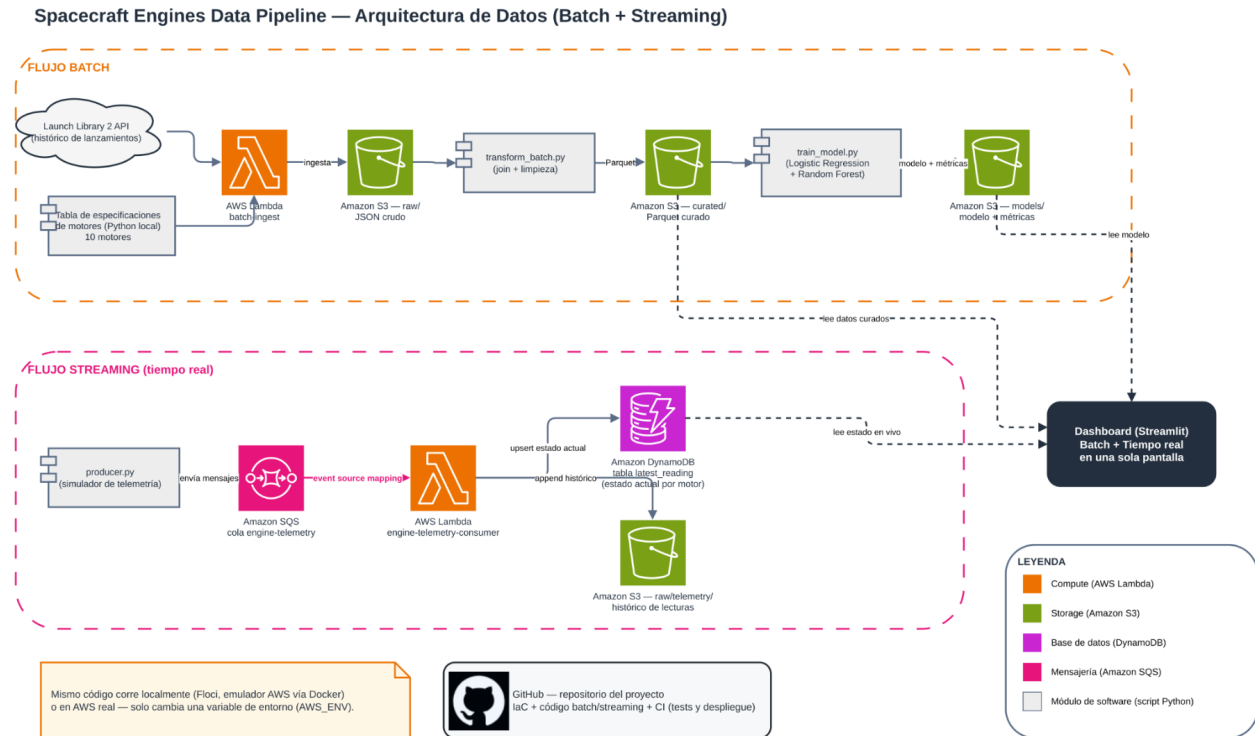


Figura 1: Arquitectura general del pipeline – flujo batch (arriba) y flujo streaming (abajo), convergiendo en el dashboard.

4. Flujo Batch

4.1. Ingesta

La ingesta batch corre como una función **AWS Lambda real** (`batch-ingest`), desplegada mediante `infra/deploy_ingest_lambda.py`. Esta Lambda:

- Consulta Launch Library 2 una vez por cada familia de cohete conocida.
- Escribe la tabla de especificaciones de motores y los lanzamientos crudos (JSON sin procesar) al bucket S3, bajo `raw/`.
- Es idempotente: si una familia ya fue ingerida, la omite en corridas posteriores en lugar de volver a consultar la API.

4.2. Transformación

El script `transform_batch.py` lee los archivos JSON crudos desde S3, los combina (*join*) con la tabla de especificaciones por familia de cohete, limpia inconsistencias de la fuente (por ejemplo, la búsqueda difusa de la API mezclaba resultados de familias distintas – el caso de Atlas V se excluye explícitamente por esta razón, documentado en el propio código), y escribe el resultado curado en formato **Parquet** a S3 -- `curated/`.

4.3. Modelo de Machine Learning

`train_model.py` entrena dos modelos (Regresión Logística y Random Forest) para predecir el éxito o fallo de un lanzamiento individual, a partir de las especificaciones del motor y el año del lanzamiento. Los resultados y modelos entrenados se guardan en `S3 -- models/`.

Resultado honesto: la señal predictiva a nivel de lanzamiento individual es débil (ROC-AUC $\approx 0,64$), y la variable `year` domina la importancia de features (85 %) – ya que las especificaciones del motor son constantes dentro de cada familia de cohete. Sin embargo, existe un hallazgo descriptivo más sólido: **la tasa de éxito varía genuinamente por familia de cohete** (desde 100 % en Falcon Heavy hasta 92 % en Saturn V), lo cual se reporta con transparencia en el dashboard en lugar de sobrevender el resultado del modelo predictivo.

5. Flujo Streaming (tiempo real)

5.1. Simulación de telemetría

`producer.py` simula lecturas de telemetría (empuje, presión de cámara, temperatura, vibración, flujo de combustible) para cada uno de los 10 motores. Los valores **no son aleatorios**: se derivan de física real – por ejemplo, el flujo de combustible se calcula con la ecuación del cohete ($F = I_{sp} \cdot g_0 \cdot \dot{m}$) usando el empuje e impulso específico reales de cada motor. Ocasionalmente se inyecta una anomalía deliberada (un pico o caída en un campo), y cada inyección queda registrada como *ground truth* en S3 para poder validar el detector después.

5.2. Procesamiento en tiempo real

Cada lectura se envía a una cola real de **Amazon SQS** (`engine-telemetry`). Esa cola dispara automáticamente – mediante un *event source mapping*, sin necesidad de invocación manual ni *polling* – una función **AWS Lambda real** (`engine-telemetry-consumer`). Esta Lambda:

- Mantiene estadísticas móviles (media y varianza) por motor usando el **algoritmo de Welford**, con el estado persistido en DynamoDB entre invocaciones.
- Calcula un *z-score* por campo y marca una lectura como anómala si supera un umbral.
- Escribe el estado actual (*upsert*) en **DynamoDB** (tabla `latest_reading`, un registro por motor) y archiva el lote crudo completo en **S3** (`raw/telemetry/`), esta última de forma histórica y sin sobrescritura.

5.3. Validación del detector de anomalías

`validate_anomaly_detector.py` compara las anomalías detectadas por la Lambda contra el *ground truth* inyectado por el propio simulador, obteniendo **95 % de recall y 76 % de precisión** – una validación real y medible, no una suposición de que el detector funciona correctamente.

6. Entorno de desarrollo local (Floci)

Todo el desarrollo y las pruebas se realizaron primero contra **Floci**, un emulador de AWS local (tipo LocalStack) ejecutado vía Docker Compose, sin necesidad de credenciales reales ni costo. Esto permitió iterar rápidamente y validar toda la lógica antes de desplegar contra la cuenta real de AWS.

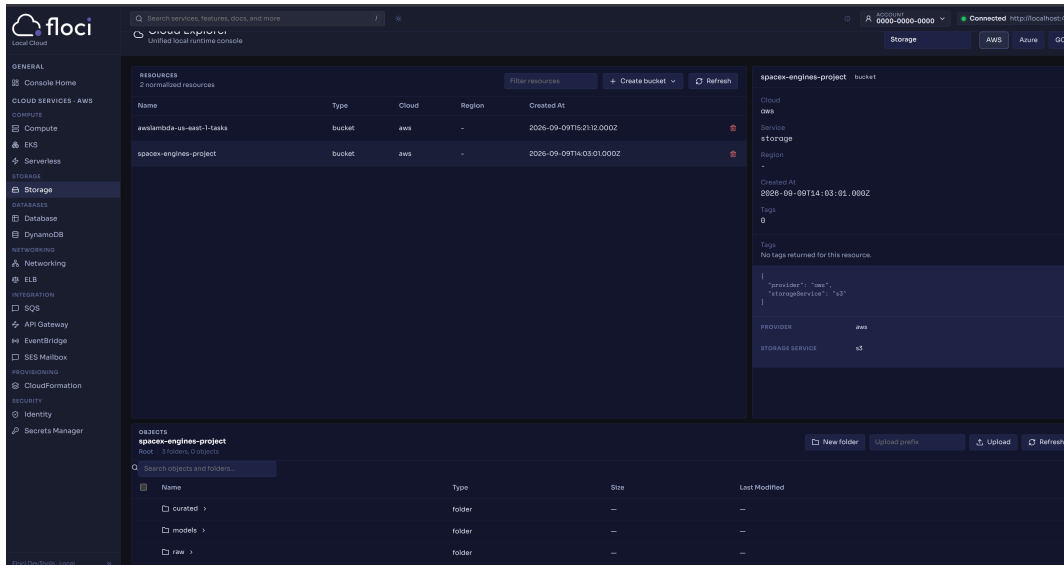


Figura 2: Bucket S3 local en Floci, con las carpetas curated/, models/ y raw/ ya pobladas.

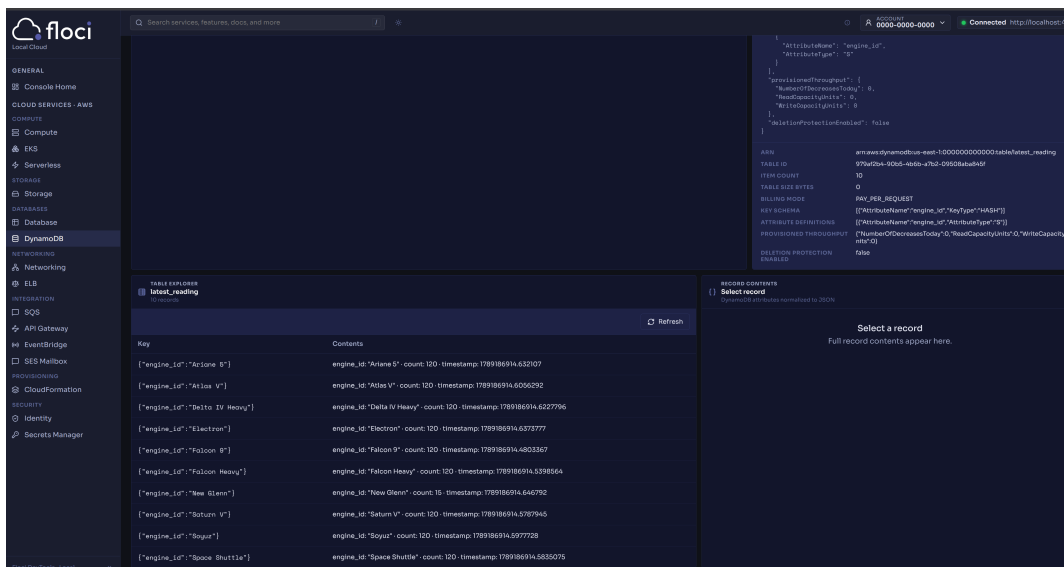


Figura 3: Tabla DynamoDB latest_reading en Floci – 10 registros, uno por motor (incluye New Glenn, agregado posteriormente).

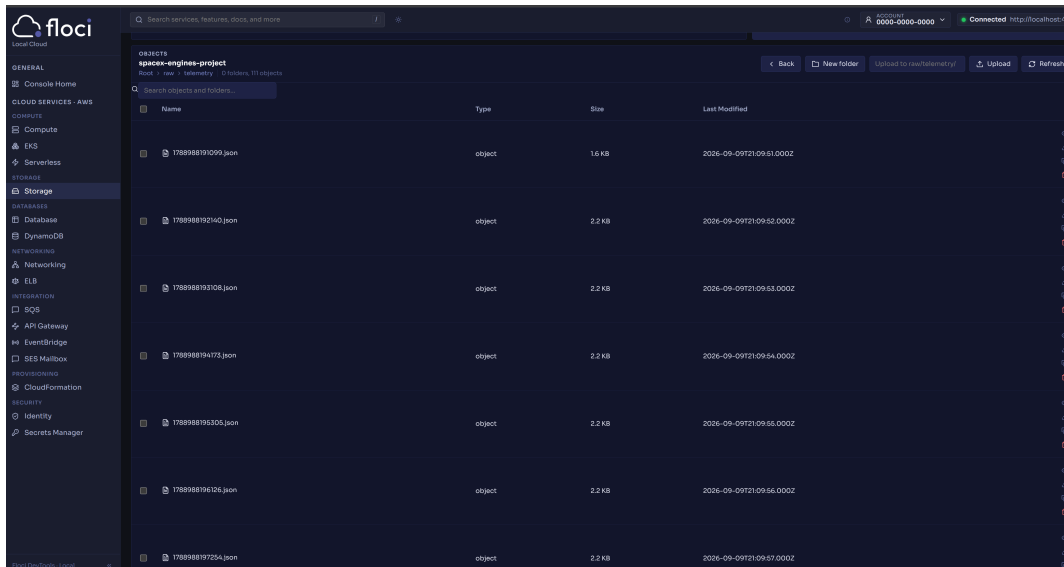


Figura 4: Histórico completo de lecturas de telemetría archivadas en S3 (`raw/telemetry/`) – 111 objetos, cada uno correspondiente a una invocación de la Lambda consumidora.

7. Despliegue en AWS real

Una vez validado localmente, el pipeline completo se desplegó contra una cuenta real de AWS, usando un usuario IAM dedicado y automatización vía GitHub Actions (ver Sección 8).

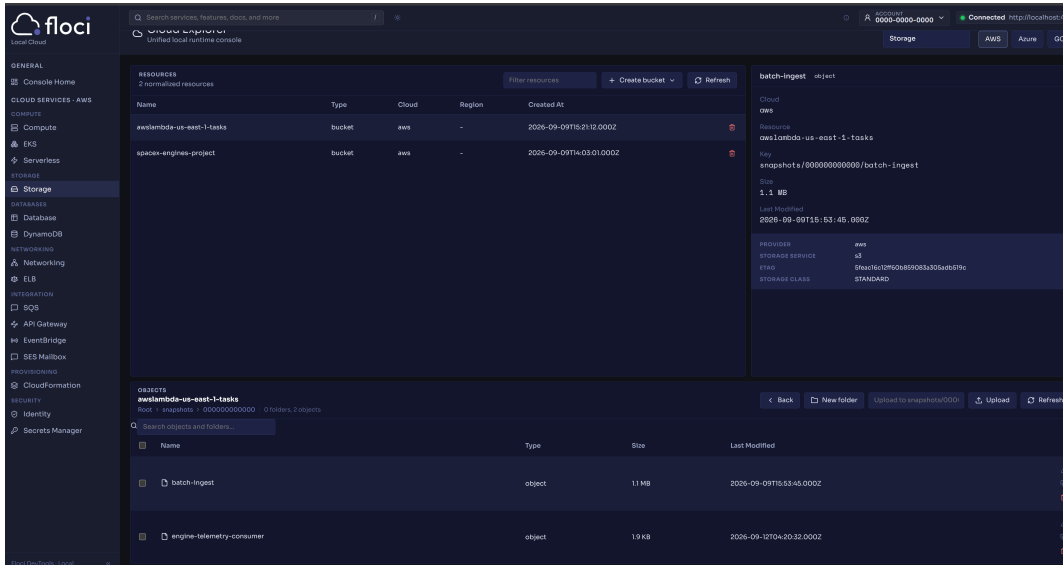


Figura 5: Paquetes de despliegue de las funciones Lambda (`batch-ingest`, `engine-telemetry-consumer`) almacenados internamente por Floki – evidencia de que corren como funciones Lambda reales, no como scripts que simulan serlo.

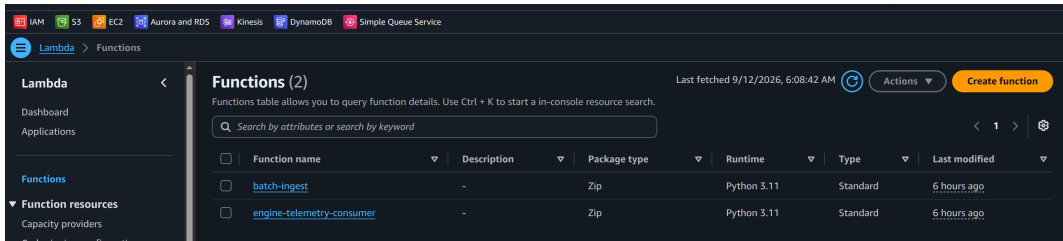


Figura 6: Las dos funciones Lambda reales desplegadas en la consola de AWS: `batch-ingest` y `engine-telemetry-consumer`.

8. CI/CD – Automatización con GitHub Actions

El despliegue y la generación de datos de prueba contra AWS real están automatizados mediante dos *workflows* de GitHub Actions, ambos de disparo manual (`workflow_dispatch`):

- **Deploy to real AWS** – crea la infraestructura (bucket S3, tabla DynamoDB, cola SQS), despliega ambas funciones Lambda, ejecuta la ingesta batch, la transformación y el entrenamiento del modelo.
- **Run streaming producer (real AWS)** – ejecuta el simulador de telemetría contra la cola SQS real, para poblar DynamoDB con datos en vivo bajo demanda (por ejemplo, justo antes de una demostración).

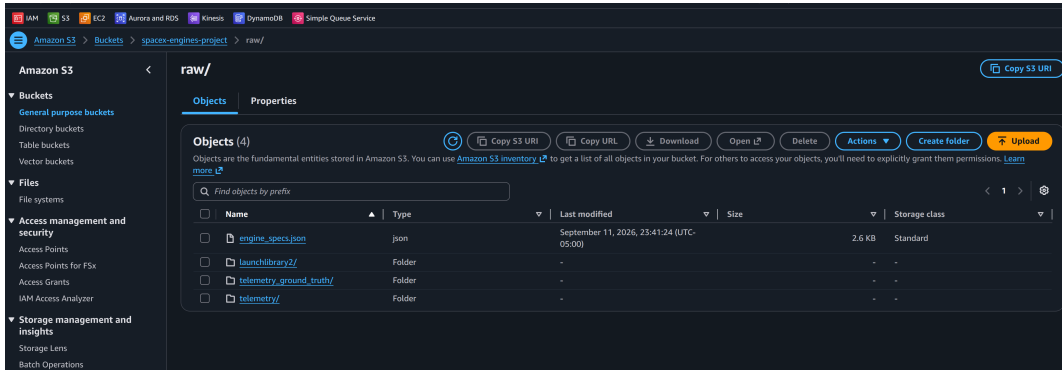


Figura 7: Bucket S3 real en AWS, con los datos crudos y curados del flujo batch.

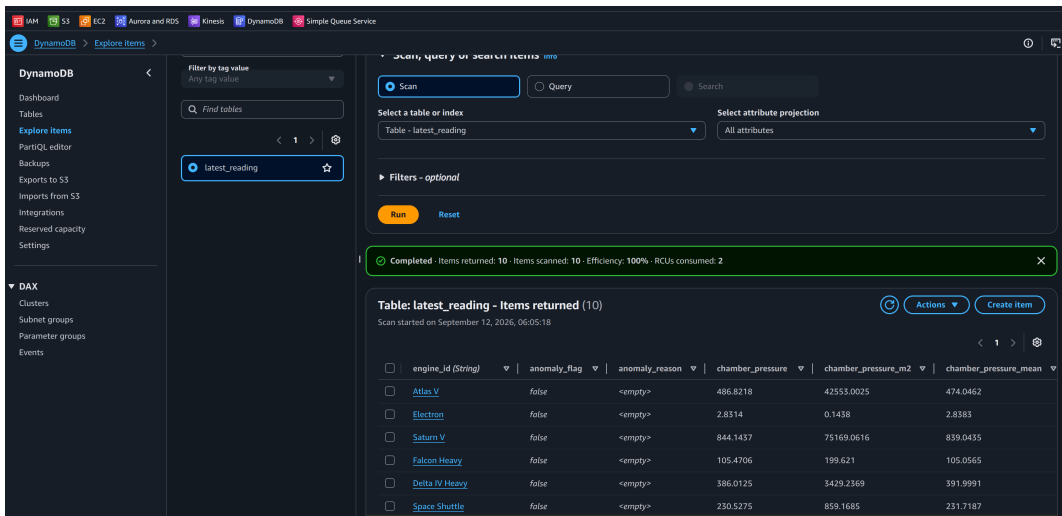


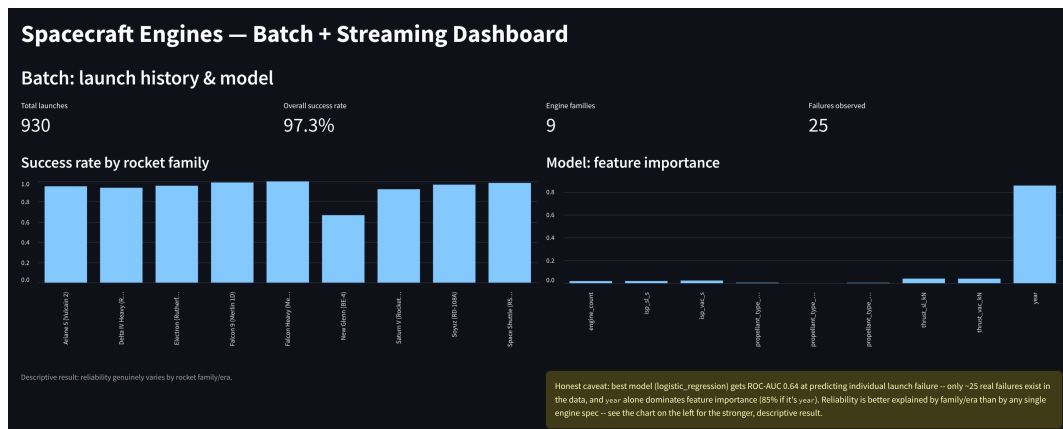
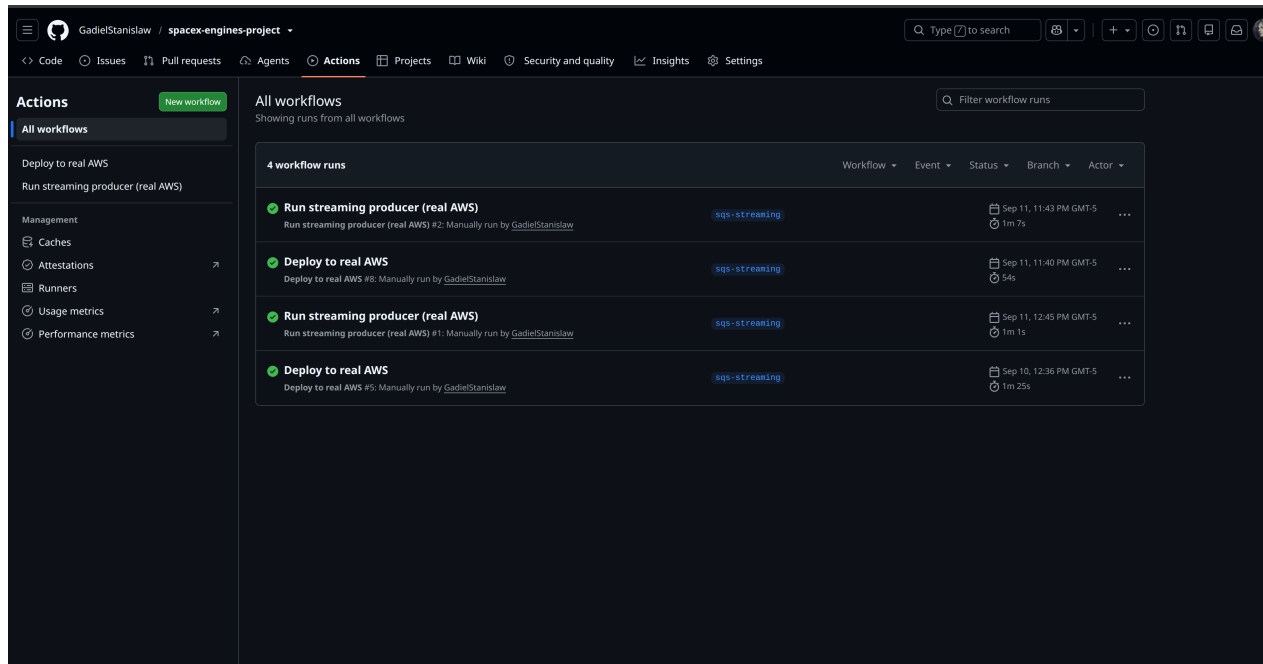
Figura 8: Tabla DynamoDB latest_reading en AWS real – 10 elementos retornados, confirmando que el flujo streaming completo (SQS → Lambda → DynamoDB) funciona de punta a punta en producción.

9. Dashboard

src/dashboard/app.py, construido con **Streamlit**, consolida ambos flujos en una sola pantalla:

- **Parte superior (batch):** métricas generales, gráfico de tasa de éxito por familia de cohete, e importancia de features del modelo – incluyendo la limitación honesta del modelo predictivo.
- **Parte inferior (streaming):** tabla en vivo con el estado actual de cada motor, actualizándose automáticamente cada 3 segundos, resaltando en rojo los motores marcados como anómalos.

El mismo código del dashboard corre sin modificaciones contra Floci o contra AWS real.



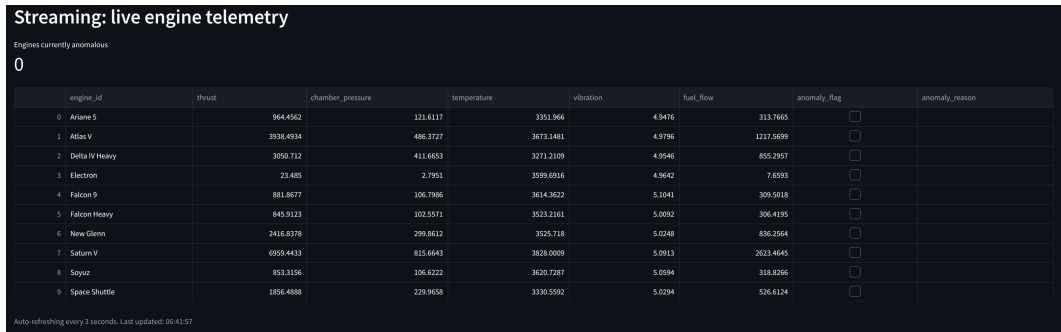


Figura 11: Parte inferior del dashboard – tabla de telemetría en vivo con los 10 motores (incluyendo New Glenn), 0 anomalías activas en este instante.

10. Tecnologías utilizadas

Categoría	Tecnología
Cómputo	AWS Lambda (2 funciones reales)
Mensajería	Amazon SQS
Base de datos	Amazon DynamoDB
Almacenamiento	Amazon S3
Seguridad	AWS IAM
CI/CD	GitHub Actions
Entorno local	Floci (emulador AWS vía Docker)
Lenguaje	Python (boto3, pandas, scikit-learn, requests)
Formato de datos	JSON (crudo), Parquet (curado)
Visualización	Streamlit
Control de versiones	Git / GitHub

Cuadro 1: Resumen de tecnologías por categoría.

11. Conclusiones

El proyecto cumple con los requisitos del curso: ingesta desde múltiples fuentes, dos tipos de flujo (batch y streaming en tiempo real), procesamiento en tiempo real con detección de anomalías validada, almacenamiento en múltiples capas (cruda y curada), y despliegue real en la nube – no simulado. El desarrollo local reproducible con Floci permitió iterar de forma rápida y sin costo antes de desplegar contra AWS real, y todo el código, tests y automatización de despliegue están versionados en el repositorio de GitHub del proyecto.